

Designing and Analysis of Algorithms

Course Code: ECS 5101/CS514

- Dr Rahul Mishra
- IIT Patna

- **Lecture 13**

Representing Binary Trees in Memory :

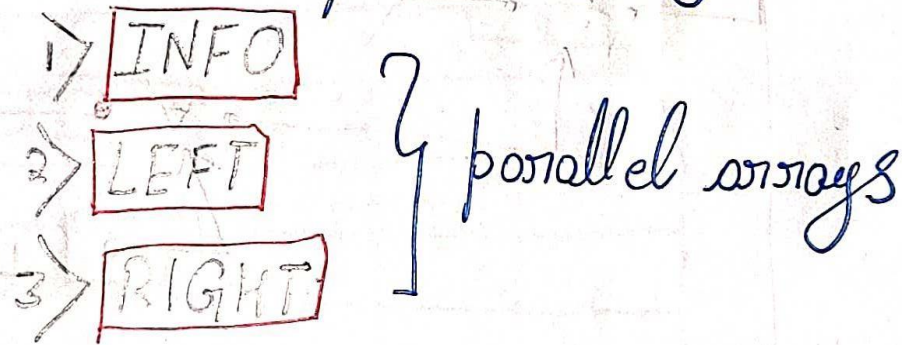
⑨

① Linked represent.

② Sequential representation using single array.

> Let T denotes a binary tree.

> Unless otherwise stated or implied, T will be maintained in memory by means of a linked representation which uses three parallel arrays

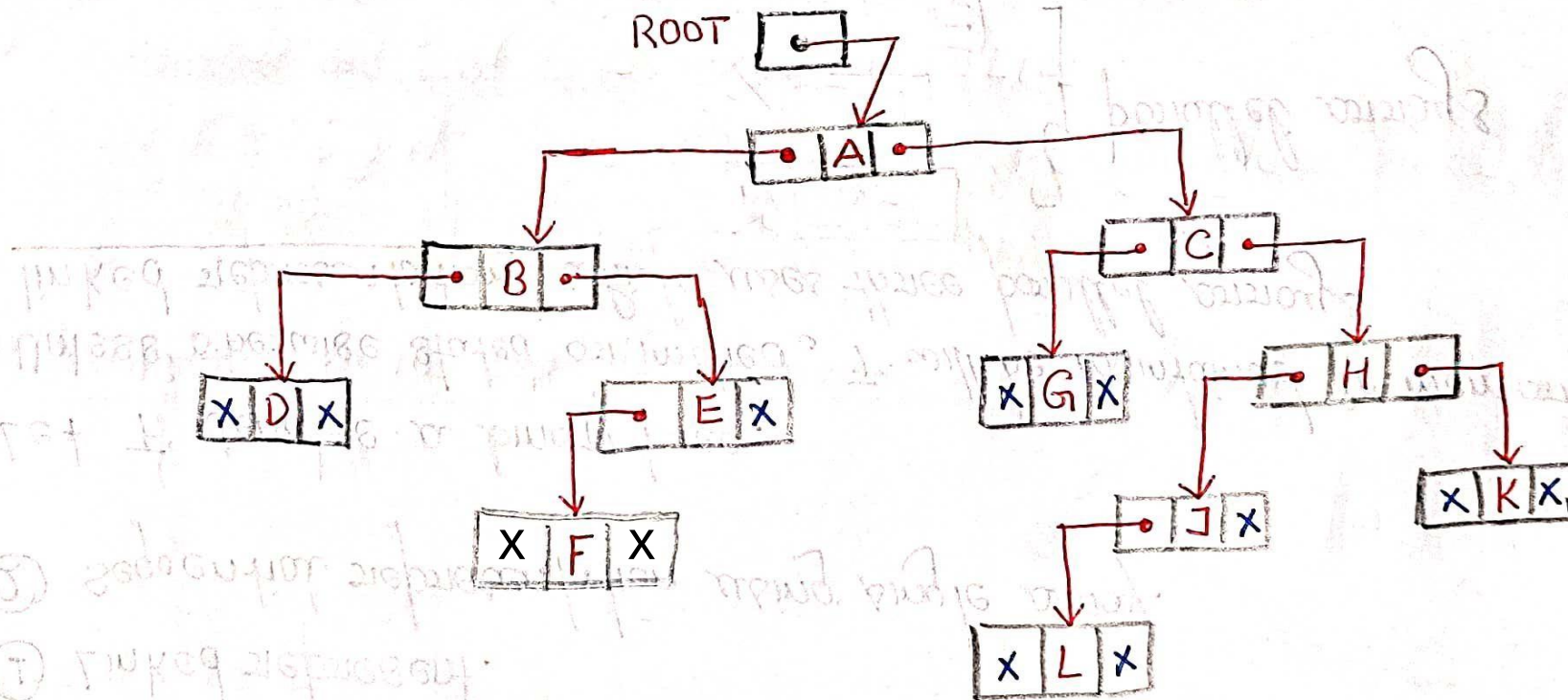


and one pointer variable "ROOT"

Each node (N) of (T) will correspond to a location (K) such that:

(10)

- (1) $INFO[K]$ contains the data of the node N
- (2) $LEFT[K]$ contains the location of the left child of node N
- (3) $RIGHT[K]$ contains the location of the right child of node N .



ROOT
5

AVAIL
8

	INFO	LEFT	RIGHT
1	K	0	0
2	C	3	6
3	G	0	0
4		14	
5	A	10	2
6	H	17	1
7	L	0	0
8		8 9	
9		4	
10	B	18	13 + 1
11		19	
12	F	0	0
13	E	12	0
14		15	
15		16	
16		17 11	
17	J	7	0
18	D	0	0
19		20	
20		0	

Sequential Representation of Binary Trees:

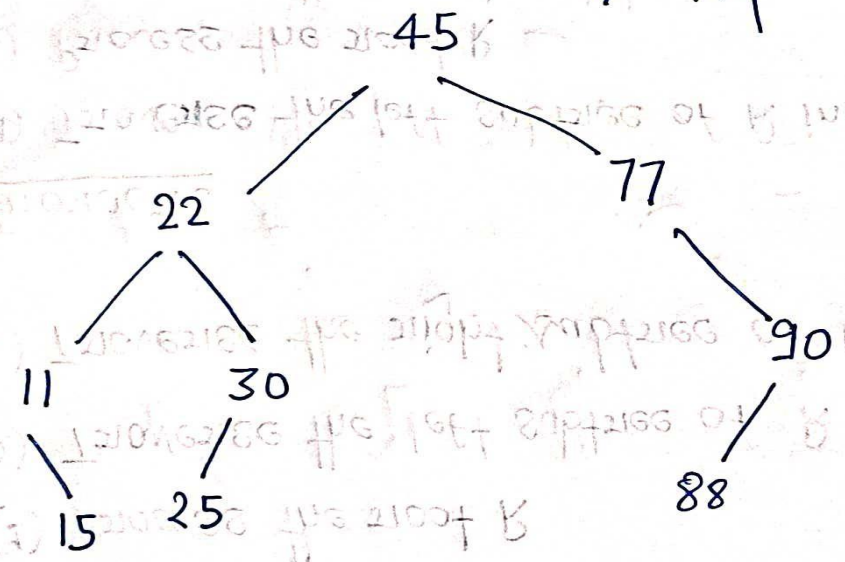
- > Suppose T is a binary tree that is complete or nearly complete.
- > An effective way to maintain T in memory called the sequential representation of T .
- > This representation uses only a single linear array $TREE$ as follows:

(a) The root R of T is stored in $TREE[1]$

(b) If a node N occupies $TREE[K]$, then its left child is stored in $TREE[2 * K]$ and its right child is stored in $TREE[2 * K + 1]$

* NULL is used to indicate an empty subtree. Particular, $TREE[1] = \text{NULL}$ indicate that the tree is empty.

Representation



TREE	
1	45
2	22
3	77
4	11
5	30
6	
7	90
8	
9	15
10	25
11	
12	
13	
14	88
15	
16	
17	
...	
29	

Observation:

- > We require 14 locations in the array TREE even though T has only 9 nodes.
- > In fact, if we included null entries for the successors of the terminal nodes
- > We would actually require TREE[29] for the right successor of TREE[14].
- > Specifically, the sequential representation of a tree with depth d will require

$2^d + 1$ elements.

* Traversing Binary Trees:

- > There are three standard ways of traversing a binary tree T with root R .
- > These three algorithms, called preorder, inorder and post order

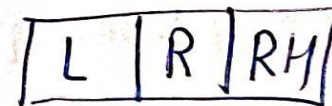
* Preorder:

- (1) Process the root R
- (2) Traverse the left subtree of R in preorder
- (3) Traverse the right subtree of R in preorder



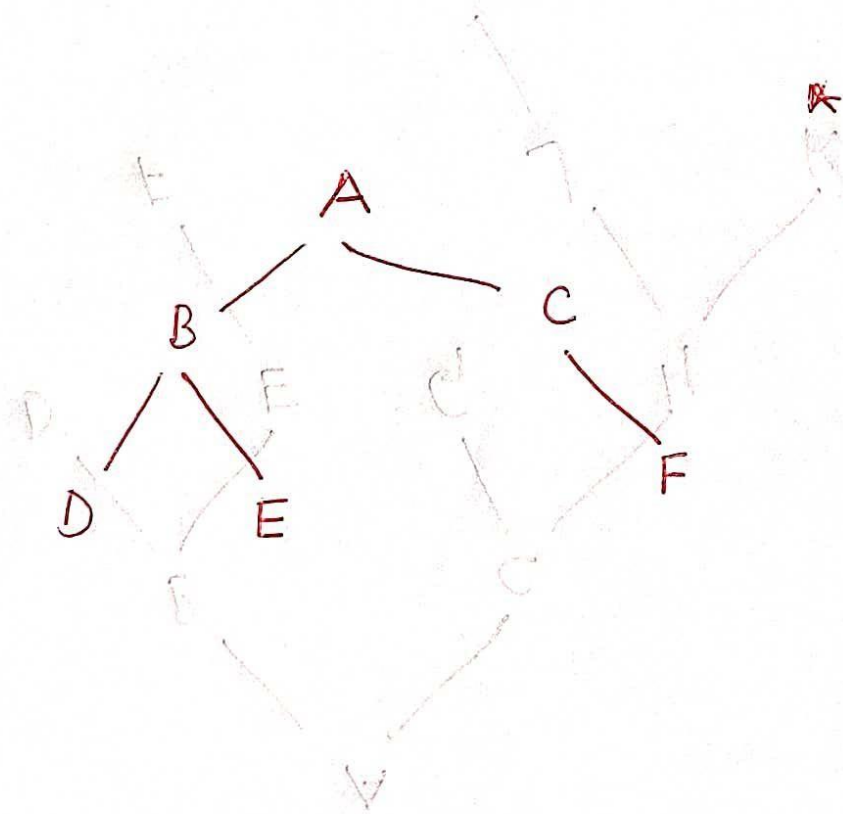
* Inorder:

- (1) Traverse the left subtree of R in inorder
- (2) Process the root R
- (3) Traverse the right subtree of R in inorder



Post order

- (1) Traverse the left subtree of R in post order
- (2) Traverse the right subtree of R in post order
- (3) Process the root R

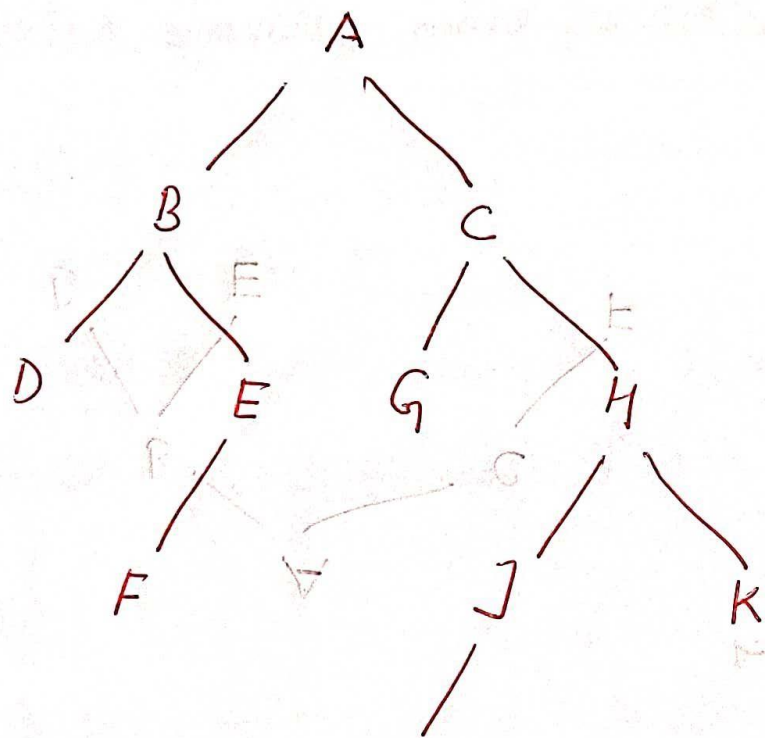


* Preorder : ABDECF

In order : DBEACF

Post order : DEBFC A

Class Assignment:



Preorder ? **ABDEFCGHLK**

Inorder ? **DBFEAGCLJHK**

Postorder ? **DFEBGLJKHCA**

Postorder: DEB E C A

Inorder: DBE A C E

Preorder: A B D E C E

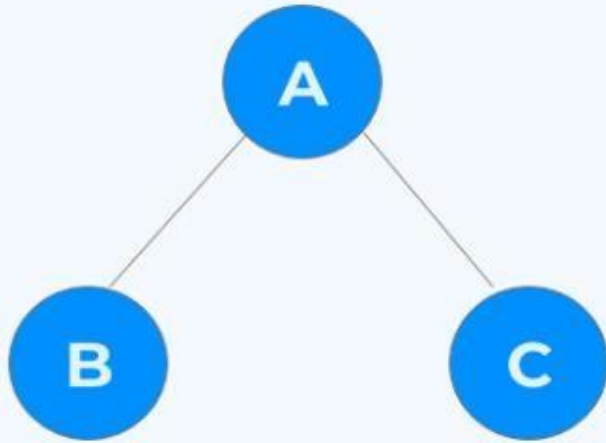
(3) Preorder: A B D E C E

(4) Inorder: D B E A C E

(5) Postorder: D F E B G L J K H C A

5 6 7 8

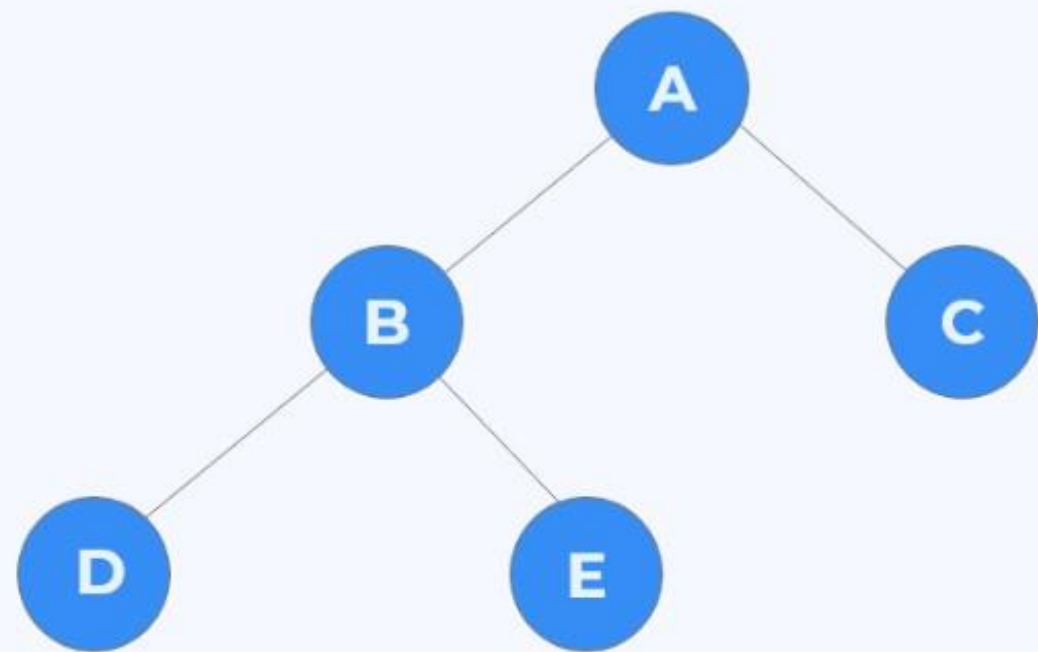
Inorder Postorder Preorder Example



PreOrder
ABC

PostOrder
BCA

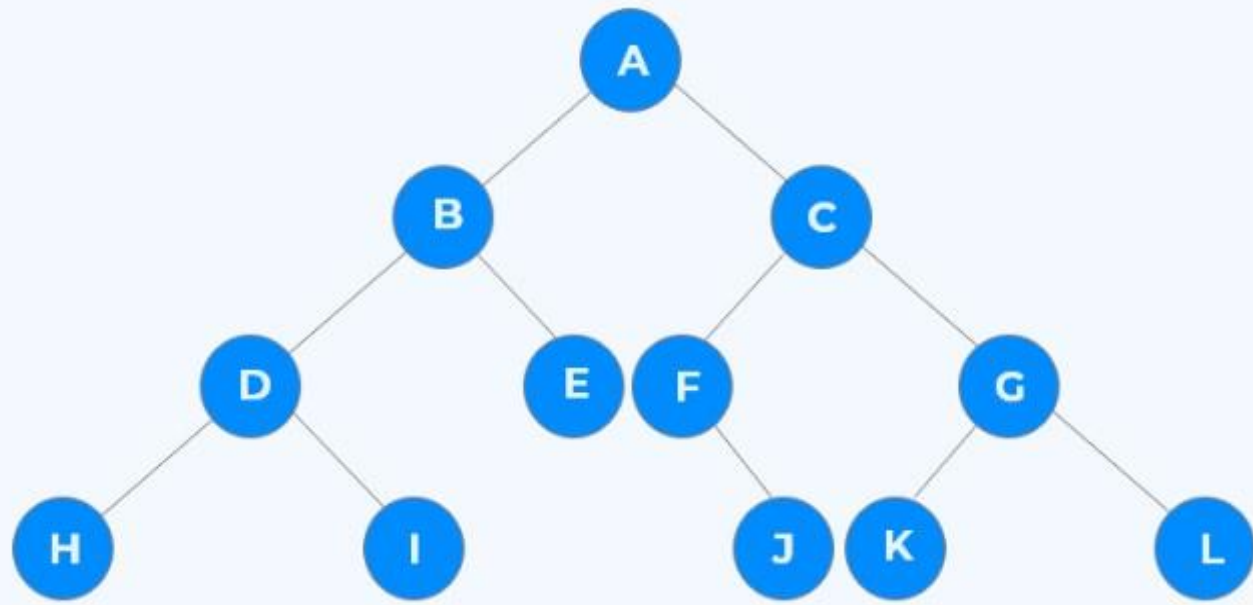
InOrder
BAC



PreOrder:
A B D E C

PostOrder:
D E B C A

Inorder:
D B E A C



PreOrder:

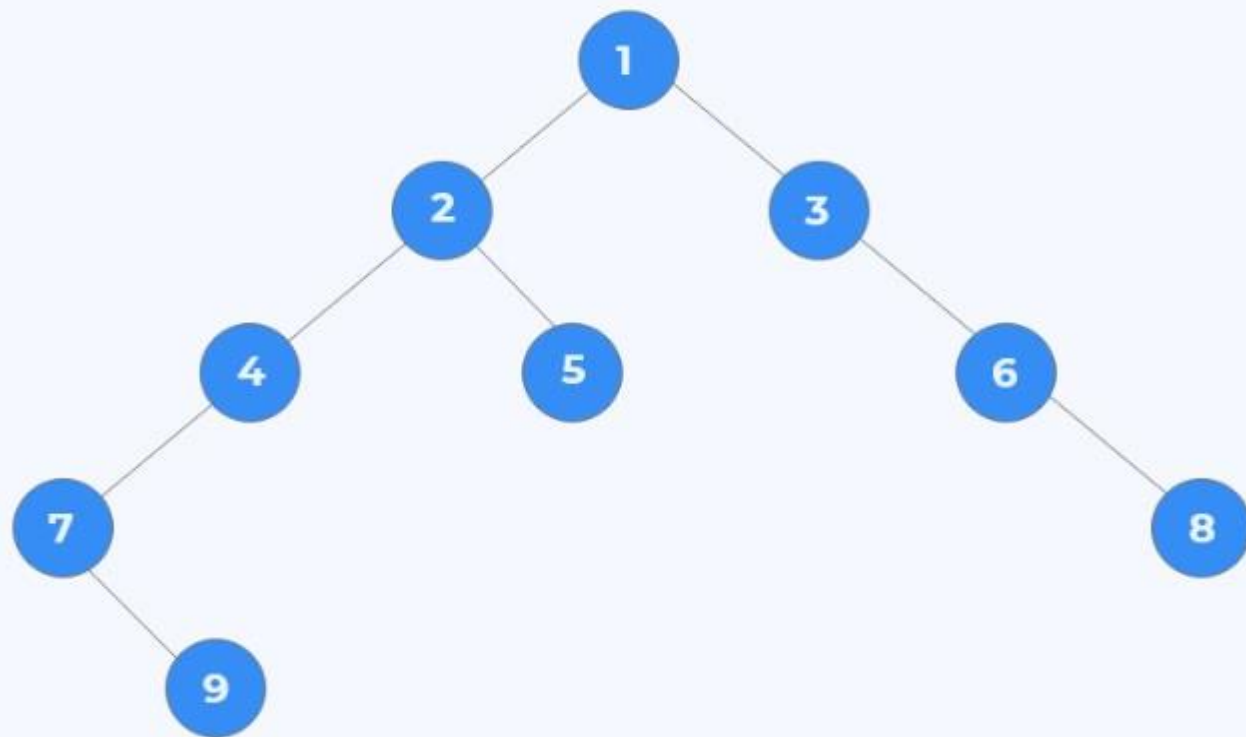
A B D H I E C F J G K L

PostOrder:

H I D E B J F K L G C A

Inorder:

H D I B E A F J C K G L



PreOrder:

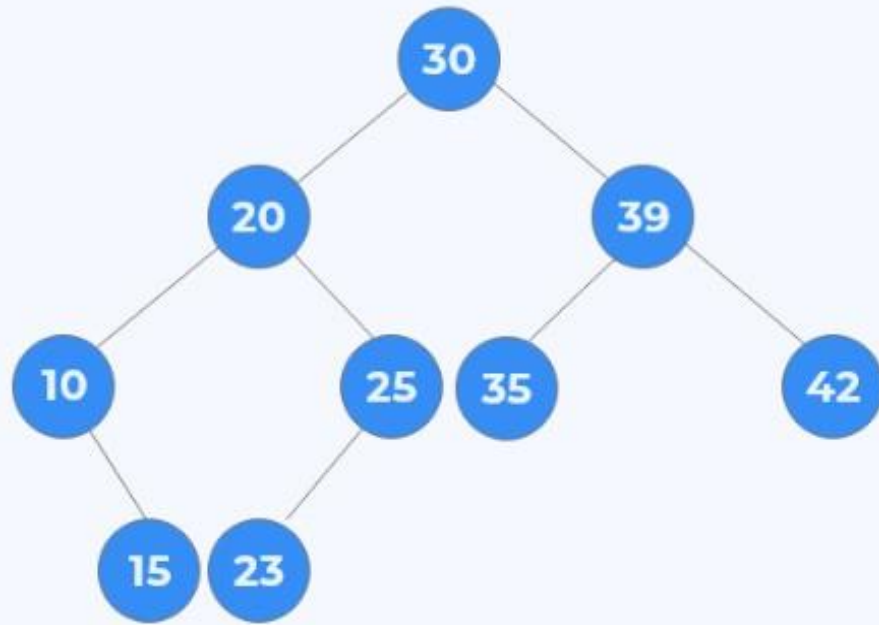
1 2 4 7 9 5 3 6 8

PostOrder:

9 7 4 5 2 8 6 3 1

Inorder:

7 9 4 2 5 1 3 6 8



PreOrder:

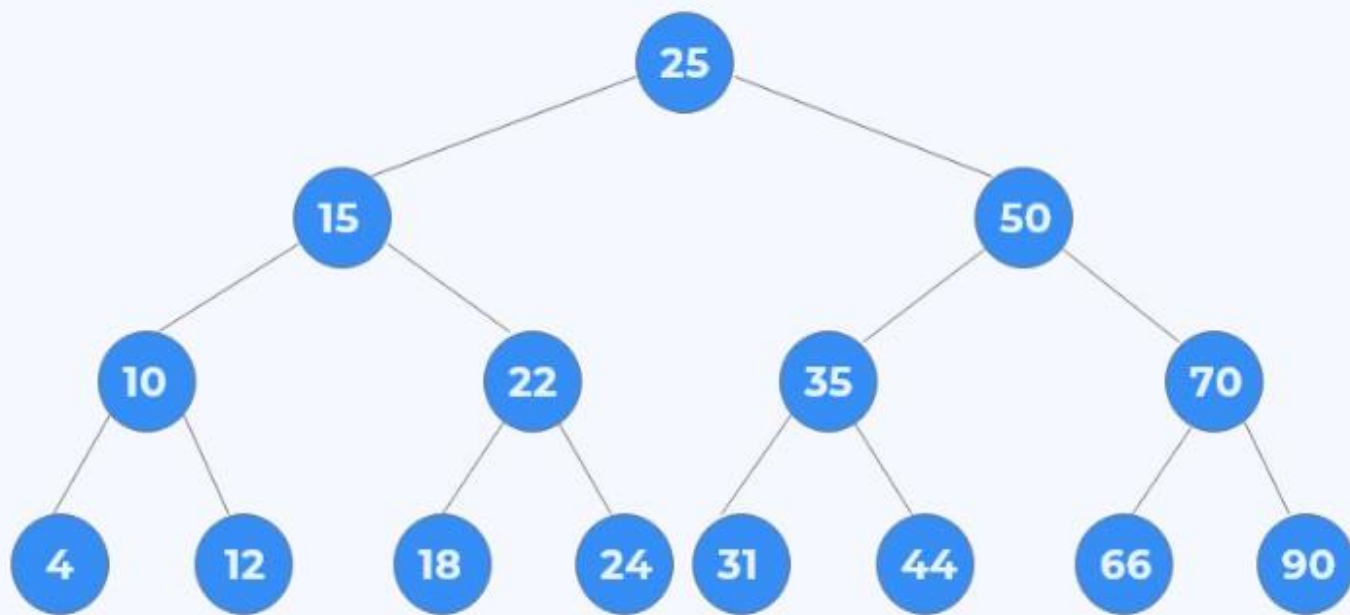
30 20 10 15 25 23 39 35 42

PostOrder:

15 10 23 25 20 35 42 39 30

Inorder:

10 15 20 23 25 30 35 39 42



PreOrder:

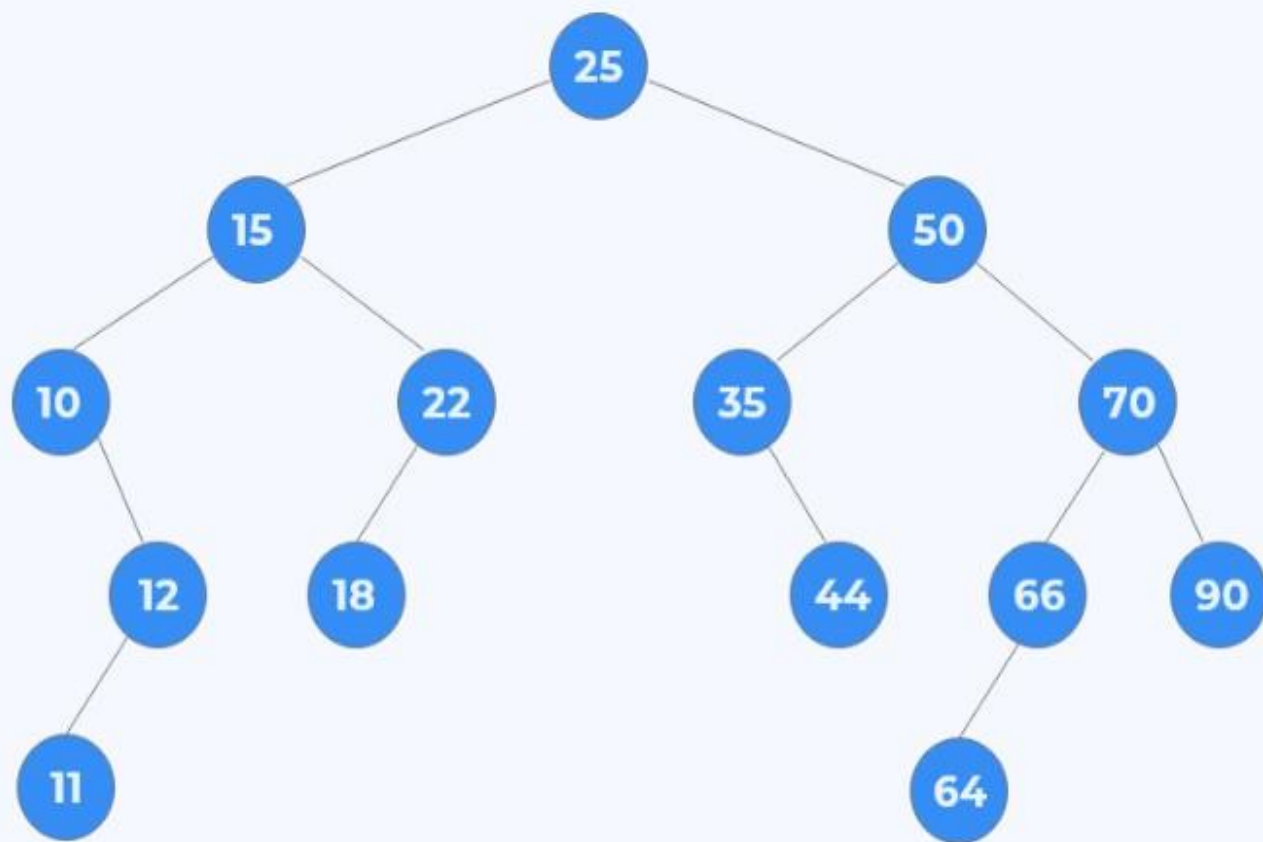
25 15 10 4 12 22 18 24 50 35 31 44 70 66 90

PostOrder:

4 12 10 18 24 22 15 31 44 35 66 90 70 50 25

Inorder:

4 10 12 15 18 22 24 25 31 35 44 50 66 70 90



PreOrder:

25 15 10 12 11 22 18 50 35 44 70 66 64 90

PostOrder:

11 12 10 18 22 15 44 35 64 66 90 70 50 25

Inorder:

10 11 12 15 18 22 25 35 44 50 64 66 70 90

BINARY SEARCH TREES :-

- One of the most important data structures in computer science.
- This structure enables one to search for and find an element with an average running time $f(n) = O(\log_2 n)$.
- This contrast with:
 - 1) Sorted linear array (binary search) $O(\log_2 n)$
 - 2) Linear linked list ($O(n)$)
- Although each node in a binary search tree may contain an entire record of data.
- The definition of the binary tree depends upon a given field whose values are distinct or may be ordered.